Lab 4: Synchronization and Channel Estimation

Name: Marc Pham

Lab 4 Task 1: Symbol Synchronization

Calculate the timing quantities, run the PSK timing experiment, and interpret BER before and after integer timing-phase recovery.

In [9]:
```
from pathlib import Path
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Image, display
import pandas as pd

def locate_lab_root():
    for base in (Path.cwd(), *Path.cwd().parents):
        if (base / "support" / "lab4_tools.py").is_file():
            return base
    raise FileNotFoundError("Open this notebook from its Lab 4 folder.")

LAB_ROOT = locate_lab_root()
sys.path.insert(0, str(LAB_ROOT / "support"))
WORK_DIR = LAB_ROOT / "work"
FIGURE_DIR = WORK_DIR / "figures"
WAVEFORM_DIR = WORK_DIR / "waveforms"
CAPTURE_DIR = Path(os.environ.get("ECE7377_LAB4_CAPTURE_DIR", WORK_DIR / "captures"))
for path in (FIGURE_DIR, WAVEFORM_DIR, CAPTURE_DIR):
    path.mkdir(parents=True, exist_ok=True)
print("Lab root:", LAB_ROOT)
print("Capture directory:", CAPTURE_DIR)
```

```
Lab root: /Users/marcpham/jupyter_notebook_files/Embedded-Wireless-Design-Labs/Lab4
Capture directory: /Users/marcpham/jupyter_notebook_files/Embedded-Wireless-Design-Labs/Lab4/work/c
aptures
```

Required calculations

In [6]:
```
modulation_orders = (4, 16, 64)
ebn0_db = 15.0
samples_per_symbol = 8
rrc_span_symbols = 10
timing_error_samples = 1

# Complete the communication-system calculations.
bits_per_symbol = {M: int(np.log2(M)) for M in modulation_orders}
esn0_db = {M: ebn0_db + 10 * np.log10(bits_per_symbol[M]) for M in modulation_orders}
rrc_tap_count = rrc_span_symbols * samples_per_symbol + 1
one_filter_delay_samples = (rrc_tap_count - 1) // 2
cascade_delay_samples = 2 * one_filter_delay_samples
timing_error_symbol_fraction = timing_error_samples / samples_per_symbol

print("bits per symbol:", bits_per_symbol)
print("Es/N0 values (dB):", esn0_db)
print("RRC taps:", rrc_tap_count)
print("one-filter and cascade delays:", one_filter_delay_samples, cascade_delay_samples)
print("one-sample error as a symbol fraction:", timing_error_symbol_fraction)
```

```
bits per symbol: {4: 2, 16: 4, 64: 6}
Es/N0 values (dB): {4: 18.010299956639813, 16: 21.020599913279625, 64: 22.78151250383644}
RRC taps: 81
one-filter and cascade delays: 40 80
one-sample error as a symbol fraction: 0.125
```

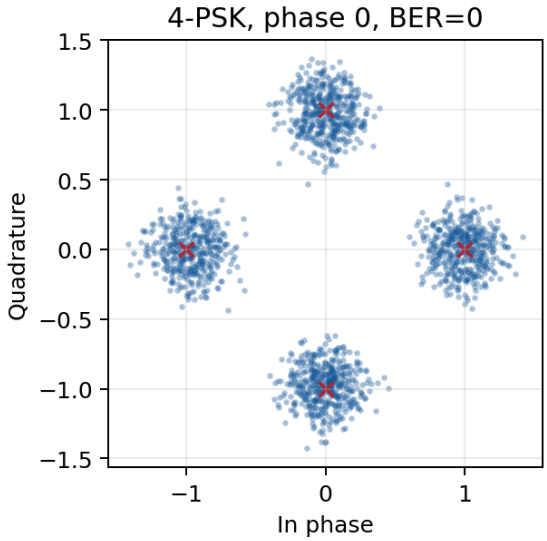
In [7]:
```python
assert bits_per_symbol == {4: 2, 16: 4, 64: 6}
assert rrc_tap_count == 81
assert one_filter_delay_samples == 40
assert cascade_delay_samples == 80
assert np.isclose(timing_error_symbol_fraction, 0.125)
print("Task 1 calculations passed.")
```

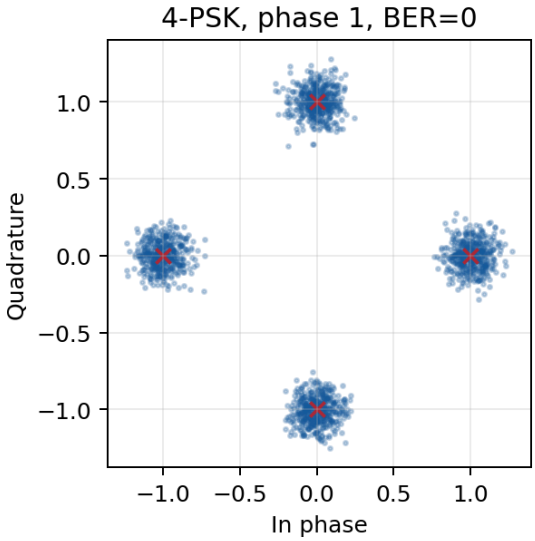
```
Task 1 calculations passed.
```

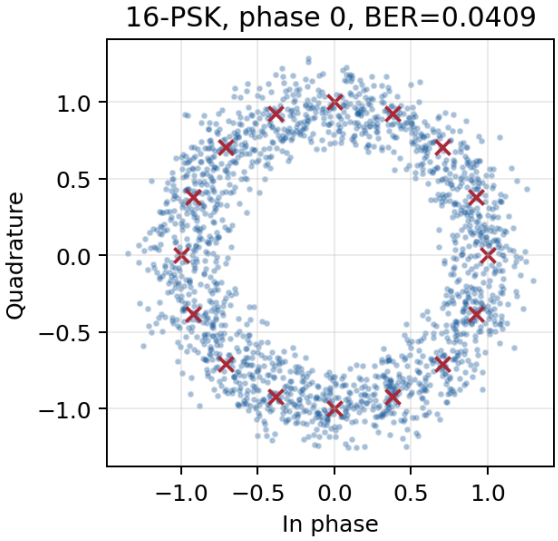
Timing study

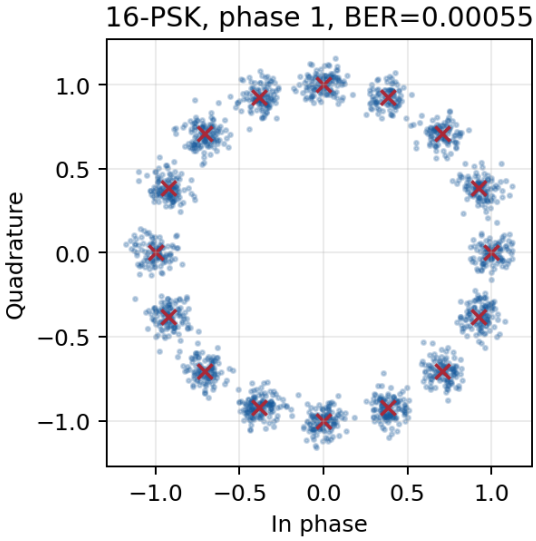
In [11]:
```python
from lab4_tools import plot_timing_study, run_timing_study, save_json

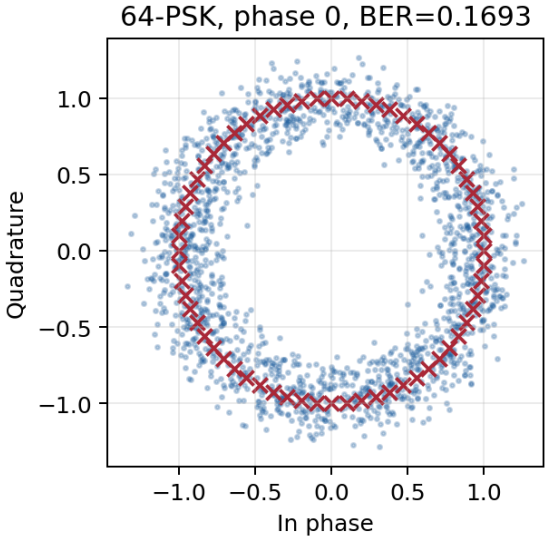
ebn0_values = (8, 12, 15, 18)
timing_errors = (0, 1, 2, 3)
study = run_timing_study(modulation_orders, ebn0_values, timing_errors)
figure_paths = plot_timing_study(study, FIGURE_DIR)
for path in figure_paths:
    display(Image(filename=path))
```

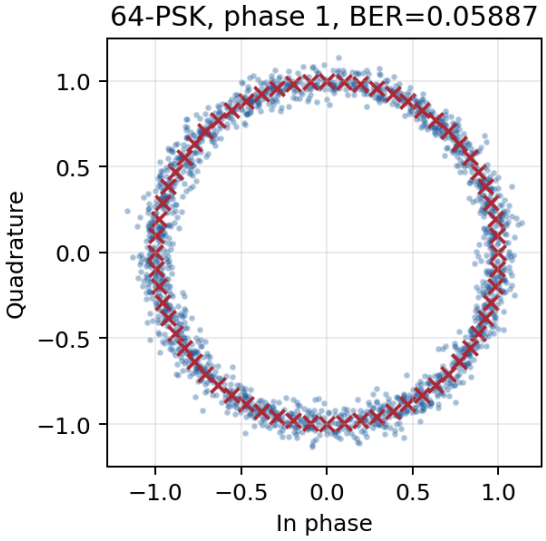

4-PSK, phase 0, BER=0
Quadrature
In phase

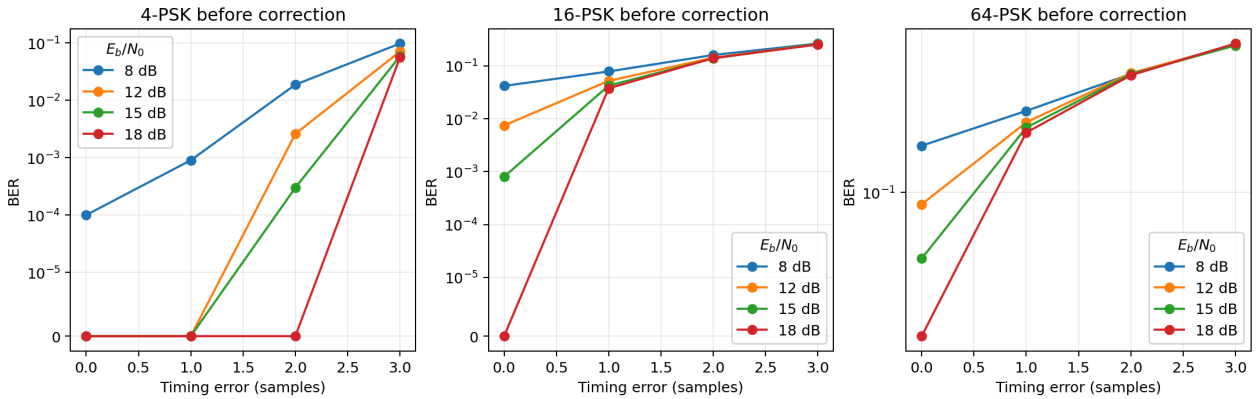

4-PSK, phase 1, BER=0
Quadrature
In phase


16-PSK, phase 0, BER=0.0409
Quadrature
In phase


16-PSK, phase 1, BER=0.00055
Quadrature
In phase


64-PSK, phase 0, BER=0.1693
Quadrature
In phase


64-PSK, phase 1, BER=0.05887
Quadrature
In phase



Report. Report the true and estimated timing phase for all three modulation orders.

In [10]:
```
timing_results_df = pd.DataFrame([
    {
        "Modulation": f"{order}-PSK",
        "True Phase": study["single"][order]["true_phase"],
        "Estimated Phase": study["single"][order]["estimated_phase"],
        "BER Before": study["single"][order]["before_ber"],
        "BER After": study["single"][order]["after_ber"],
    }
    for order in modulation_orders
])

display(timing_results_df)
```

	Modulation	True Phase	Estimated Phase	BER Before	BER After
0	4-PSK	1	1	0.000000	0.000000
1	16-PSK	1	1	0.040900	0.000550
2	64-PSK	1	1	0.169333	0.058867

Report. Explain why timing error produces intersymbol interference. Explain why an off-center sampling instant introduces intersymbol interference.

Timing error causes the receiver to sample the pulse away from its ideal symbol center. At the correct sampling instant, the pulse-shaping and matched filters are designed so that neighboring symbols contribute little or no interference. When sampling occurs off-center, the neighboring pulses are no longer at their zero-crossings, so parts of adjacent symbols contribute to the measured value. This overlap between symbols produces intersymbol interference (ISI) and distorts the received constellation.

Report. Explain why higher-order PSK is more sensitive. Explain why increasing PSK order reduces tolerance to timing error and noise.

Higher-order PSK places more constellation points around the same unit circle, so the angular separation between neighboring symbols becomes smaller. As a result, timing error and noise require less distortion to move a received symbol across a decision boundary. This makes higher-order PSK more likely to decode a symbol incorrectly for the same amount of timing offset or noise. Therefore, increasing the PSK order improves bits per symbol but reduces tolerance to synchronization errors and noise.

Report. Separate the effect of increasing bits per symbol M from the effect of the associated increase in bits per symbol at fixed Eb/N0. Separate the effect of increasing bits per symbol from decreasing angular spacing.

Increasing M increases the number of bits carried by each PSK symbol, since $k = \log_2(M)$. At a fixed E_b/N_0, this also increases E_s/N_0 because each symbol contains more bits and therefore more energy. However, increasing M also places more constellation points around the same circle, reducing the angular spacing between neighboring

symbols. This smaller angular separation is what makes higher-order PSK more sensitive to noise and timing errors, even though the energy per symbol increases.

Report. Discuss how BER changes with the sweep over the timing error and the SNR E_b/N_0.

BER increases with timing error because sampling farther from the ideal symbol center introduces more ISI, while BER decreases as the SNR E_b/N_0 increases because the signal becomes more robust to noise. The lowest BER is therefore expected at high E_b/N_0 and small timing errors, while lower E_b/N_0 and larger timing errors produce worse performance. At $E_b/N_0 = 15$ dB with a one-sample timing error, synchronization reduced BER from 0 to 0 for 4-PSK, 0.0409 to 0.00055 for 16-PSK, and 0.16933 to 0.05887 for 64-PSK. These results show that timing synchronization improves BER, although higher-order PSK remains more sensitive to the combined effects of noise and timing error.

Lab 4 Task 2: OFDM Phase Offset

Complete the OFDM accounting and pilot phase calculation, then measure BER and FER from 0 through 90 degrees.

In [1]:
```
from pathlib import Path
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import Image, display

def locate_lab_root():
    for base in (Path.cwd(), *Path.cwd().parents):
        if (base / "support" / "lab4_tools.py").is_file():
            return base
    raise FileNotFoundError("Open this notebook from its Lab 4 folder.")

LAB_ROOT = locate_lab_root()
sys.path.insert(0, str(LAB_ROOT / "support"))
WORK_DIR = LAB_ROOT / "work"
FIGURE_DIR = WORK_DIR / "figures"
WAVEFORM_DIR = WORK_DIR / "waveforms"
CAPTURE_DIR = Path(os.environ.get("ECE7377_LAB4_CAPTURE_DIR", WORK_DIR / "captures"))
for path in (FIGURE_DIR, WAVEFORM_DIR, CAPTURE_DIR):
    path.mkdir(parents=True, exist_ok=True)
print("Lab root:", LAB_ROOT)
print("Capture directory:", CAPTURE_DIR)
```

```
Lab root: /Users/marcpham/jupyter_notebook_files/Embedded-Wireless-Design-Labs/Lab4
Capture directory: /Users/marcpham/jupyter_notebook_files/Embedded-Wireless-Design-Labs/Lab4/work/c
aptures
```

OFDM calculations

In [2]:
```
nfft = 64
cyclic_prefix_samples = 16
pilot_subcarriers = 4
qpsk_bits_per_symbol = 2

# Derive the active-carrier and frame counts.
active_subcarriers = 52
data_subcarriers = active_subcarriers - pilot_subcarriers
bits_per_frame = data_subcarriers * qpsk_bits_per_symbol
time_samples_per_frame = nfft + cyclic_prefix_samples

print("data subcarriers:", data_subcarriers)
print("bits per frame:", bits_per_frame)
print("time samples per frame:", time_samples_per_frame)
```

```
data subcarriers: 48
bits per frame: 96
time samples per frame: 80
```

In [4]:
```python
assert data_subcarriers == 48
assert bits_per_frame == 96
assert time_samples_per_frame == 80
print("OFDM accounting passed.")
```

```
OFDM accounting passed.
```

Pilot phase correction check

In [5]:
```python
known_pilots = np.ones(4, dtype=complex)
received_pilots = known_pilots * np.exp(1j * np.deg2rad(30))

# Estimate and remove the common pilot phase.
phase_hat_rad = np.angle(np.sum(np.conj(known_pilots) * received_pilots))
corrected_pilots = received_pilots * np.exp(-1j * phase_hat_rad)

print("estimated phase (degrees):", np.rad2deg(phase_hat_rad))
print("largest corrected pilot error:", np.max(np.abs(corrected_pilots - known_pilots)))
```

```
estimated phase (degrees): 29.999999999999996
largest corrected pilot error: 7.437084071668725e-18
```

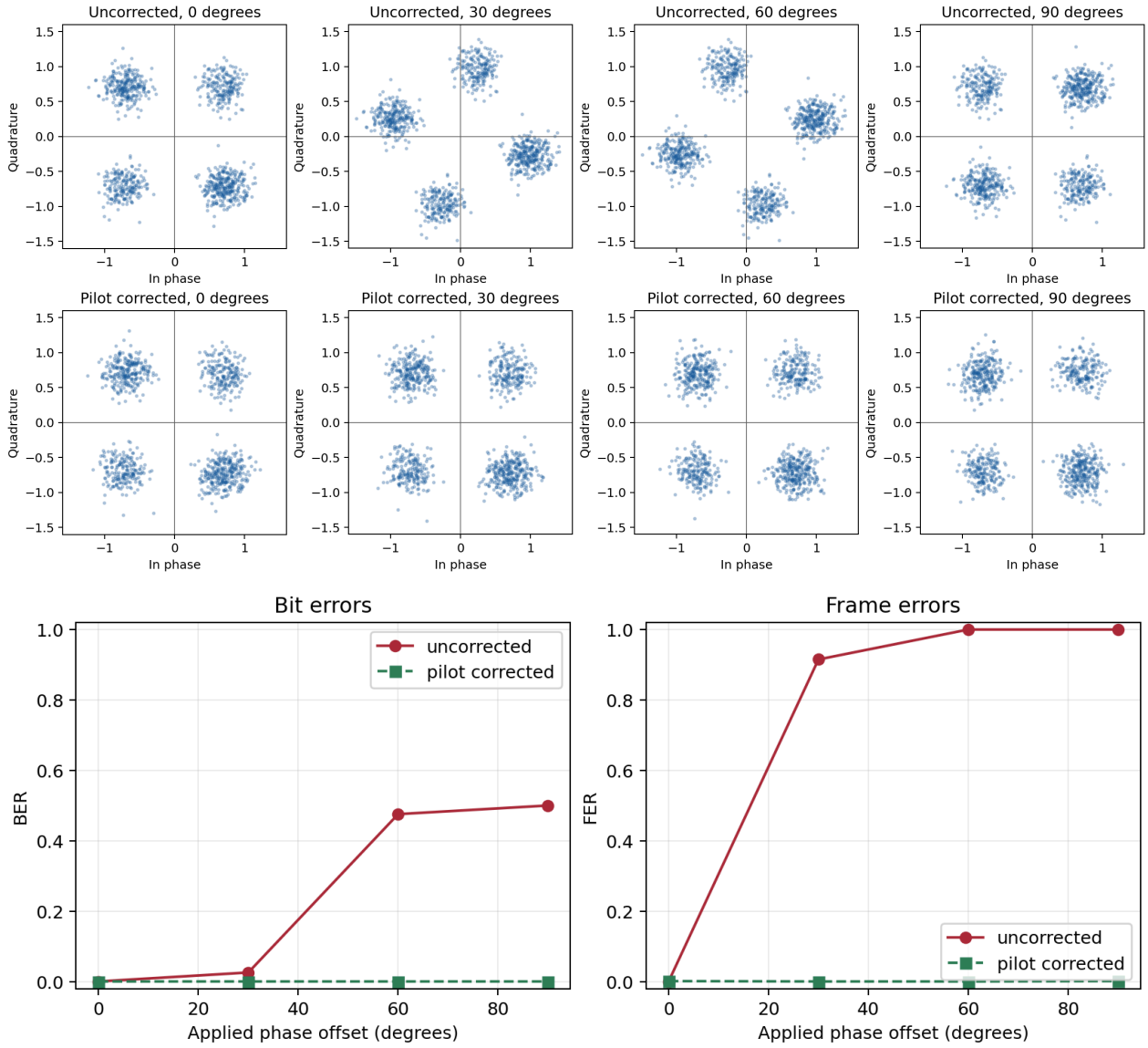
In [6]:
```python
assert np.isclose(np.rad2deg(phase_hat_rad), 30.0)
assert np.max(np.abs(corrected_pilots - known_pilots)) < 1e-12
print("Pilot correction passed.")
```

```
Pilot correction passed.
```

Full phase sweep

In [11]:
```python
from lab4_tools import ofdm_reference, plot_ofdm_phase_study

result = ofdm_reference(
    phase_degrees=(0, 30, 60, 90),
    frames=1000,
    ebn0_db=10.0,
    message="hello world",
)
figure_paths = plot_ofdm_phase_study(result, FIGURE_DIR)
for path in figure_paths:
    display(Image(filename=path))
```



Report. Report BER and FER before correction at all four phase offsets. Report the mean pilot phase estimate and corrected error rates BER and FER. Include both saved figures and the result table.

In [14]:
```
phase_results_df = pd.DataFrame([{
    "Phase Offset (deg)": row["phase_degrees"], "BER": row["ber"], "FER": row["fer"],
    "Mean Pilot Estimated Phase  (deg)": row["estimated_phase_mean_degrees"],
    "Corrected BER": row["corrected_ber"], "Corrected FER": row["corrected_fer"],
} for row in result["rows"] ])

display( phase_results_df.style.format({"BER": "{:.6f}", "FER": "{:.3f}", "Mean Pilot Estimated Pha
                                        "Corrected BER": "{:.7f}", "Corrected FER": "{:.3f}"}))
```

	Phase Offset (deg)	BER	FER	Mean Pilot Estimated Phase (deg)	Corrected BER	Corrected FER
0	0	0.000000	0.000	0.156070	0.0000104	0.001
1	30	0.025531	0.915	30.162642	0.0000000	0.000
2	60	0.475469	1.000	60.118485	0.0000000	0.000
3	90	0.500000	1.000	90.050035	0.0000104	0.001

Report. Explain why FER may approach one while BER remains much smaller.

FER can approach one even when BER is much smaller because a frame is counted as incorrect if it contains even one bit error. Each frame contains 96 bits, so even a relatively small BER can make it likely that at least one of those

96 bits is wrong. For example, at 30° phase offset, the BER is only 0.0255, but the FER is already 0.915, meaning 91.5% of frames contain at least one error. At 60° and 90°, the BER rises to about 0.475 and 0.500, respectively, causing the FER to reach 1.0, so every frame contains at least one error.

Report. Explain the difference between constant phase offset and CFO. Explain why a constant phase rotates every subcarrier similarly, while CFO produces phase that changes with sample or OFDM-symbol index.

A constant phase offset applies the same fixed rotation to the entire received OFDM symbol, so all subcarriers are rotated by approximately the same angle. In contrast, carrier frequency offset (CFO) causes phase to continuously accumulate with time, so the phase changes with the sample index and from one OFDM symbol to the next. Mathematically, constant phase behaves like $e^{j\phi}$, while CFO behaves like $e^{j2\pi\Delta fn/f_s}$, which depends on sample index n. Therefore, a constant phase can be corrected with one common rotation, while CFO produces a time-varying phase error that must first be estimated and compensated.

Report. Include the OFDM grid, IFFT and FFT code.

The OFDM grid, IFFT, and FFT code are all included in the function `ofdm_phase_sweep()` in Will's support file `support/lab4_dsp.py`. The OFDM grid is output below.

In [4]:
```
from lab4_tools import get_ofdm_grid
result = get_ofdm_grid(
    phase_degrees=(0, 30, 60, 90),
    frames=1000,
    ebn0_db=10.0,
    message="hello world",
)
df2 = pd.DataFrame(result)
display(df2)
```

	0	1	2	3	4	5	6	7	8	
0	0.0+0.0j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107+0.707107j	1.0+0.0j	-0.707107-0.707107j	-0.7 0.7
1	0.0+0.0j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107+0.707107j	1.0+0.0j	-0.707107-0.707107j	-0.7 0.7
2	0.0+0.0j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107+0.707107j	1.0+0.0j	-0.707107-0.707107j	-0.7 0.7
3	0.0+0.0j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107+0.707107j	1.0+0.0j	-0.707107-0.707107j	-0.7 0.7
4	0.0+0.0j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107+0.707107j	1.0+0.0j	-0.707107-0.707107j	-0.7 0.7
...	...	...	...	...	...	...	...	...	...	
995	0.0+0.0j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107+0.707107j	1.0+0.0j	-0.707107-0.707107j	-0.7 0.7
996	0.0+0.0j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107+0.707107j	1.0+0.0j	-0.707107-0.707107j	-0.7 0.7
997	0.0+0.0j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107+0.707107j	1.0+0.0j	-0.707107-0.707107j	-0.7 0.7
998	0.0+0.0j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107+0.707107j	1.0+0.0j	-0.707107-0.707107j	-0.7 0.7
999	0.0+0.0j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107-0.707107j	0.707107-0.707107j	-0.707107+0.707107j	1.0+0.0j	-0.707107-0.707107j	-0.7 0.7

1000 rows × 64 columns

Lab 4 Task 3: PN Channel Estimation

Generate one degree-10 PN period, validate acquisition with a synthetic record, transmit it once with GNU Radio, and estimate the scalar link coefficient from a N310 capture.

In [1]:
```
from pathlib import Path
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Image, display

def locate_lab_root():
    for base in (Path.cwd(), *Path.cwd().parents):
        if (base / "support" / "lab4_tools.py").is_file():
            return base
    raise FileNotFoundError("Open this notebook from its Lab 4 folder.")

LAB_ROOT = locate_lab_root()
sys.path.insert(0, str(LAB_ROOT / "support"))
WORK_DIR = LAB_ROOT / "work"
FIGURE_DIR = WORK_DIR / "figures"
WAVEFORM_DIR = WORK_DIR / "waveforms"
CAPTURE_DIR = Path(os.environ.get("ECE7377_LAB4_CAPTURE_DIR", WORK_DIR / "captures"))
for path in (FIGURE_DIR, WAVEFORM_DIR, CAPTURE_DIR):
    path.mkdir(parents=True, exist_ok=True)
print("Lab root:", LAB_ROOT)
print("Capture directory:", CAPTURE_DIR)
```

```
Lab root: /Users/marcpham/jupyter_notebook_files/Embedded-Wireless-Design-Labs/Lab4
Capture directory: /Users/marcpham/jupyter_notebook_files/Embedded-Wireless-Design-Labs/Lab4/work/c
aptures
```

Waveform and file calculations

Report. Calculate the PN length, transmit duration, waveform byte count, receive sample count, and receive byte count.

In [2]:
```
pn_degree = 10
sample_rate_sps = 1_250_000
acquisition_lead_seconds = 2.0
edge_guard_samples = 4096
capture_seconds = 10.0

# Derive every count from the parameters above
pn_length = 2**pn_degree - 1
acquisition_lead_samples = int(acquisition_lead_seconds * sample_rate_sps)
waveform_samples = acquisition_lead_samples + pn_length + 2 * edge_guard_samples
waveform_bytes = waveform_samples * 8  # complex64 = 8 bytes/sample
waveform_duration_seconds = waveform_samples / sample_rate_sps
capture_samples = int(capture_seconds * sample_rate_sps)
capture_bytes = capture_samples * 8  # complex64 = 8 bytes/sample

print("PN length:", pn_length)
print("waveform samples and bytes:", waveform_samples, waveform_bytes)
print("waveform duration (s):", waveform_duration_seconds)
print("capture samples and bytes:", capture_samples, capture_bytes)
```

```
PN length: 1023
waveform samples and bytes: 2509215 20073720
waveform duration (s): 2.007372
capture samples and bytes: 12500000 100000000
```

In [3]:
```
assert pn_length == 1023
assert acquisition_lead_samples == 2_500_000
```

```
assert waveform_samples == 2_509_215
assert waveform_bytes == 20_073_720
assert np.isclose(waveform_duration_seconds, 2.007372)
assert capture_samples == 12_500_000
assert capture_bytes == 100_000_000
print("Waveform and capture calculations passed.")
```

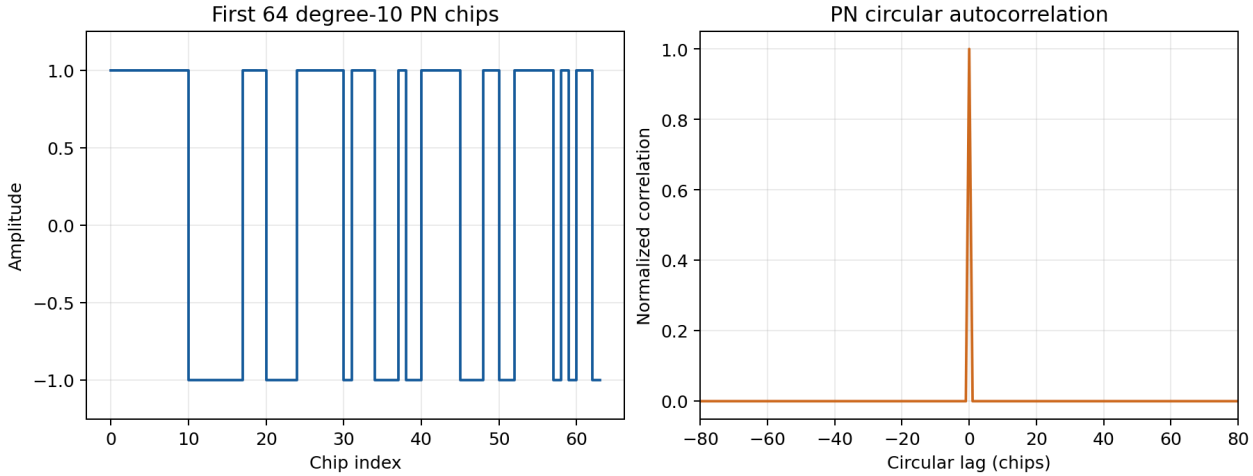
```
Waveform and capture calculations passed.
```

Generate and inspect the transmit file

In [4]:
```
from lab4_tools import generate_pn_waveform, plot_pn_sequence

reference, waveform_path = generate_pn_waveform(
    WAVEFORM_DIR / "task3_degree10_pn_burst_c64.dat"
)
sequence_figure = plot_pn_sequence(FIGURE_DIR)
print(waveform_path, waveform_path.stat().st_size, "bytes")
display(Image(filename=sequence_figure))
```

```
/Users/marcpham/jupyter_notebook_files/Embedded-Wireless-Design-Labs/Lab4/work/waveforms/task3_degr
ee10_pn_burst_c64.dat 20073720 bytes
```



Synthetic receiver check

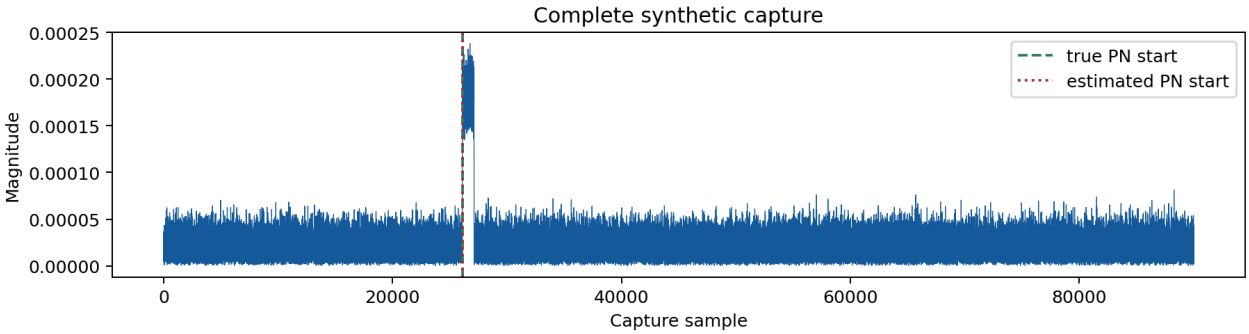
In [5]:
```
from lab4_tools import make_synthetic_pn_capture, plot_synthetic_pn

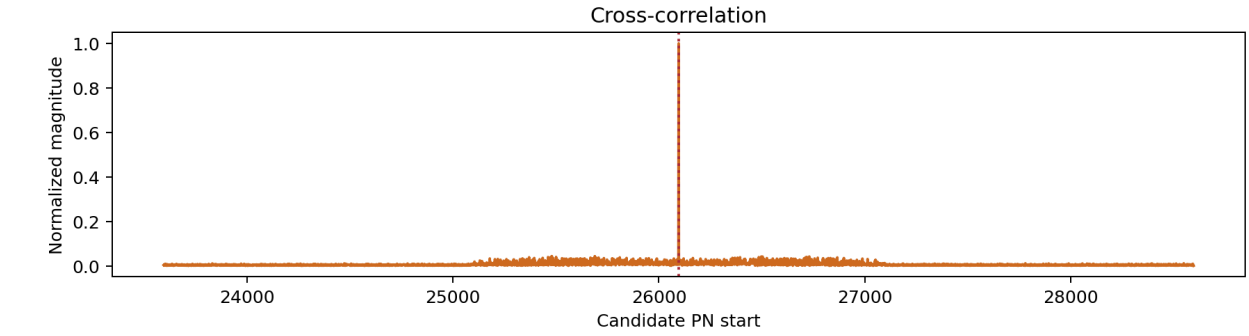
delay_samples=22_000
synthetic_capture, synthetic_result, truth = make_synthetic_pn_capture(delay_samples=22_000)

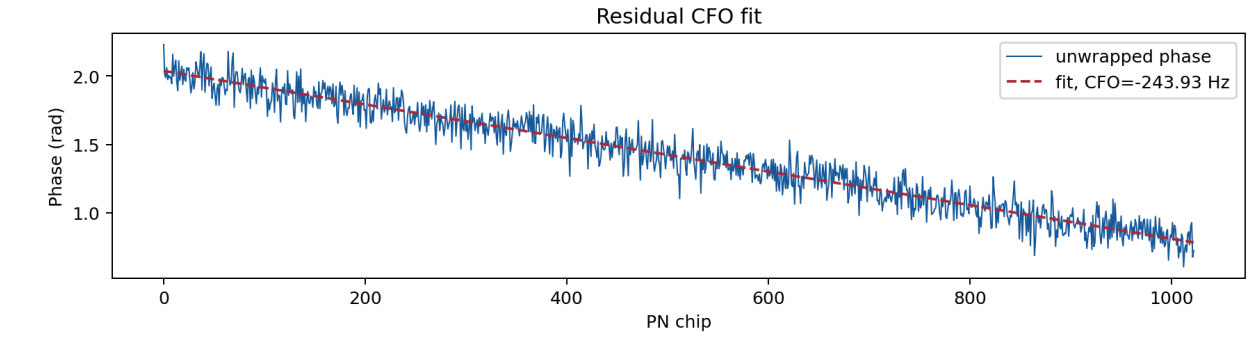
# To do: predict the PN start from the inserted delay and edge guard.
expected_start_sample = edge_guard_samples + delay_samples
assert expected_start_sample == truth["start_sample"]
assert synthetic_result["start_sample"] == expected_start_sample
print("expected and recovered start:", expected_start_sample, synthetic_result["start_sample"])
print("true and estimated CFO:", truth["cfo_hz"], synthetic_result["cfo_hz"])
print("chip error rate:", synthetic_result["chip_error_rate"])

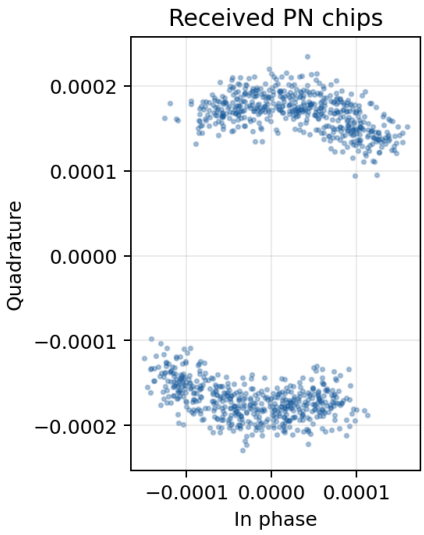
for path in plot_synthetic_pn(synthetic_capture, synthetic_result, truth, FIGURE_DIR):
    display(Image(filename=path))
```

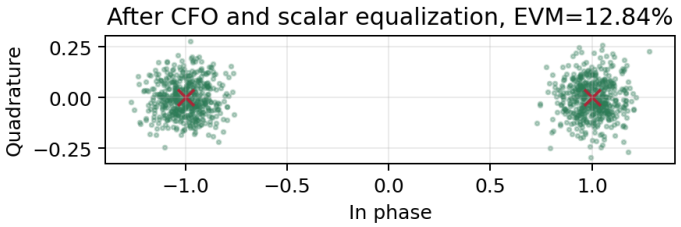
```
expected and recovered start: 26096 26096
true and estimated CFO: -245.0 -243.93136865932303
chip error rate: 0.0
```











GNU Radio run

1. On the RX server, open `flowgraphs/lab4_finite_pn_receiver.grc` and enter the assigned RX device address, subdevice, antenna, gain, output filenames, and squelch threshold.
2. On the TX server, open `flowgraphs/lab4_finite_pn_transmitter.grc` and enter the assigned TX address, subdevice, antenna, gain, and transmit filename.
3. Capture one second of noise with the transmitter stopped.
4. Start the 10-second receiver first. Start the one-shot transmitter immediately afterward.

5. Confirm 12,500,000 raw complex64 samples and 100,000,000 bytes.
6. Keep both raw and squelched files. Use the raw file for correlation and least squares.

The two-second quiet lead-in occurs inside the finite file. The PN period itself is transmitted only once.

Report. Confirm 12,500,000 raw complex64 samples and 100,000,000 bytes.

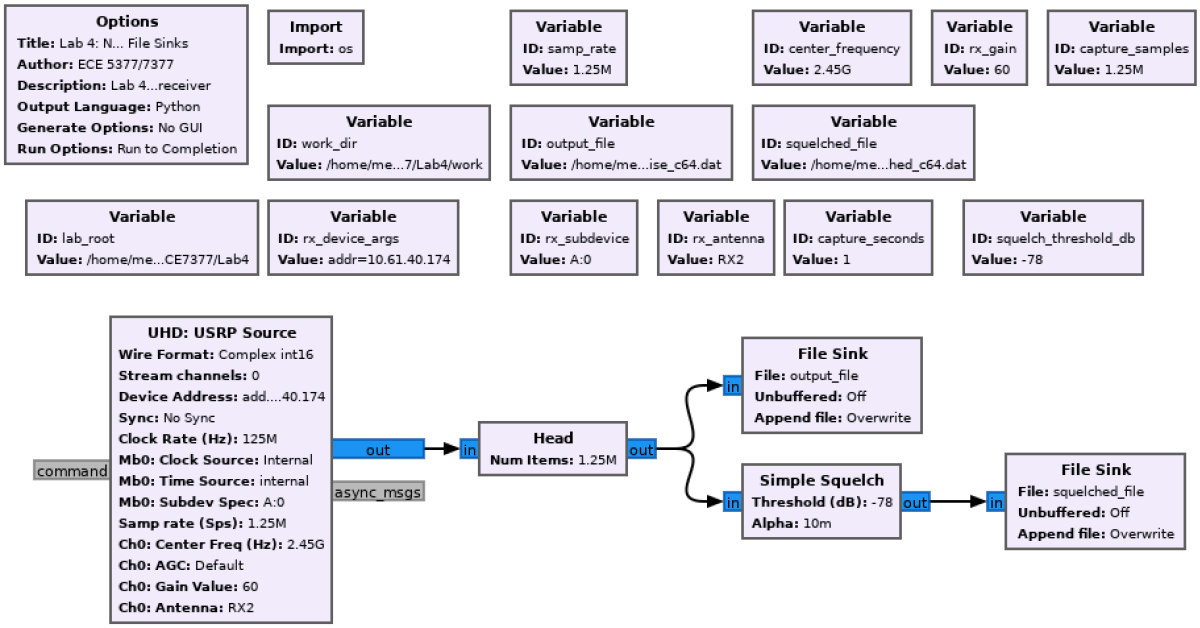
Examining the head block on the Rx flowgraph for raw and squelched files reveals the 12.5M raw complex64 samples.

In [34]:
```
!ls -la $CAPTURE_DIR/task3_gain25_run1_raw_c64.dat
!ls -la $CAPTURE_DIR/task3_gain25_run1_squelched_c64.dat
!ls -la $CAPTURE_DIR/task3_noise_c64.dat
```

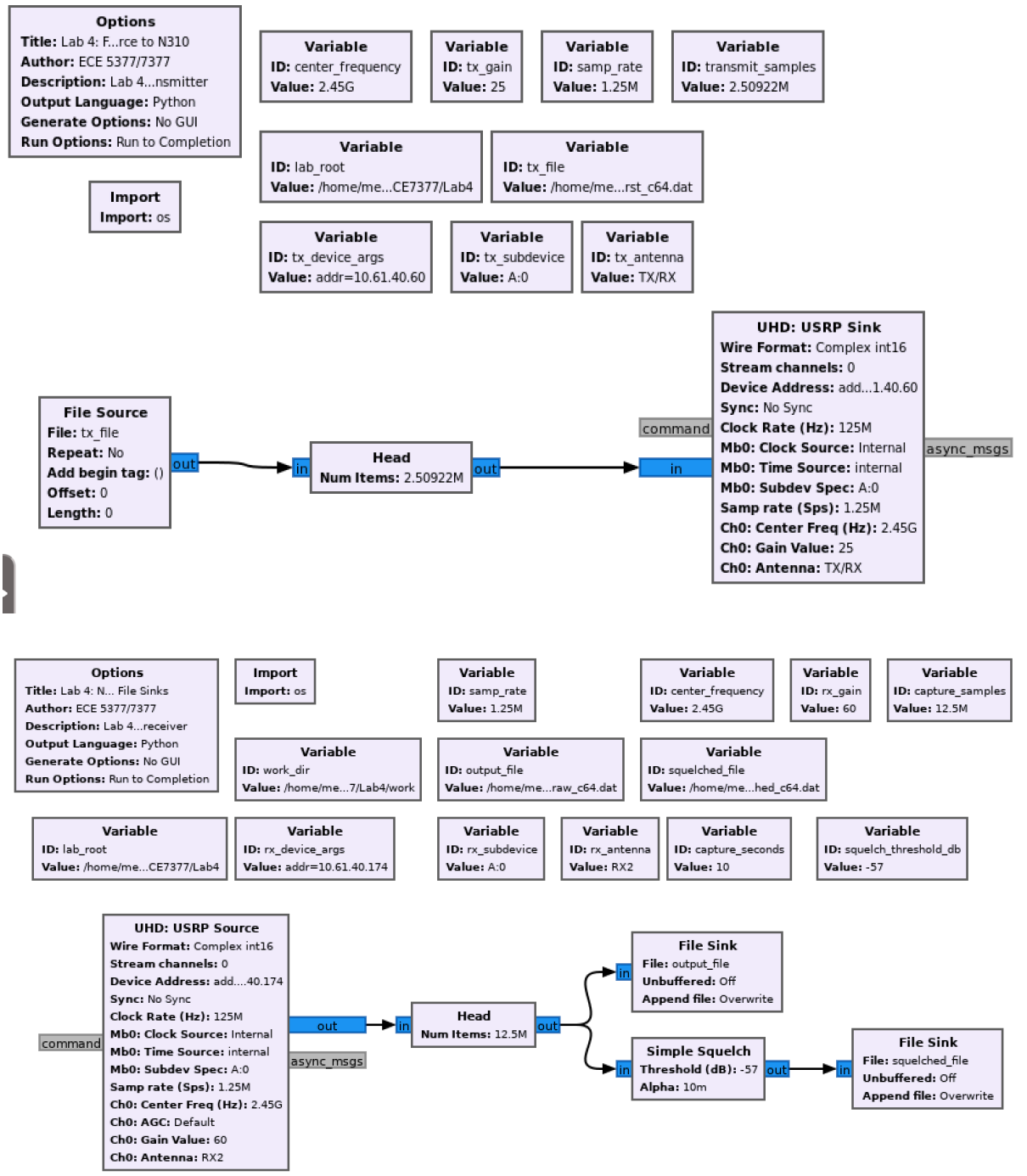
```
-rw-r--r--  1 marcpham  staff  100000000 Oct 13 09:02 /Users/marcpham/jupyter_notebook_files/Embedd
ed-Wireless-Design-Labs/Lab4/work/captures/task3_gain25_run1_raw_c64.dat
-rw-r--r--  1 marcpham  staff  100000000 Oct 13 09:01 /Users/marcpham/jupyter_notebook_files/Embedd
ed-Wireless-Design-Labs/Lab4/work/captures/task3_gain25_run1_squelched_c64.dat
-rw-r--r--  1 marcpham  staff  10000000 Oct 13 09:01 /Users/marcpham/jupyter_notebook_files/Embedde
d-Wireless-Design-Labs/Lab4/work/captures/task3_noise_c64.dat
```

Report. Include screenshots of both GNU Radio canvases and record all radio metadata.

Rx Flowgraph for Noise Capture



Tx/Rx Flowgraphs for Raw and Squelched Files



Analyze the nominal real capture

Report. Report noise and active power, squelch threshold, correlation peak sample (start sample), correlation evidence, CFO estimate, complex channel estimate $\hat{h}$, magnitude $|\hat{h}|$, phase, RMS EVM, and chip error rate.

In [25]:
```
from lab4_dsp import pn_sequence
from lab4_tools import analyze_pn_capture, plot_real_pn_measurement

raw_path = CAPTURE_DIR / "task3_gain25_run1_raw_c64.dat"
squelched_path = CAPTURE_DIR / "task3_gain25_run1_squelched_c64.dat"
noise_path = CAPTURE_DIR / "task3_noise_c64.dat"
if not raw_path.is_file():
    raise FileNotFoundError(f"Run GNU Radio first. Missing {raw_path}")
```

```python
record = analyze_pn_capture(
    raw_path,
    noise_path=noise_path,
    squelched_path=squelched_path,
)
training = pn_sequence(10)
corrected = np.asarray(record["corrected"])

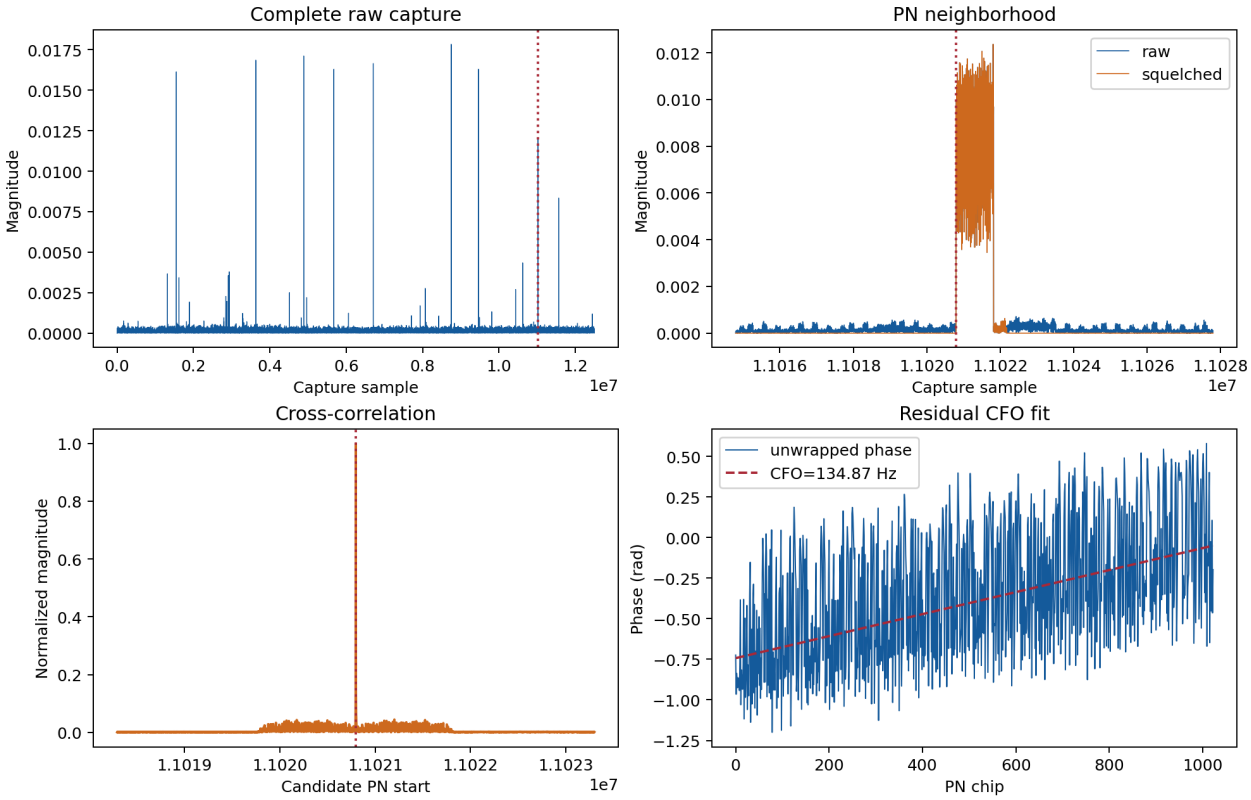
# Calculate scalar least squares and equalize.
channel_estimate = np.vdot(training, corrected) / np.vdot(training, training)
equalized = corrected / channel_estimate

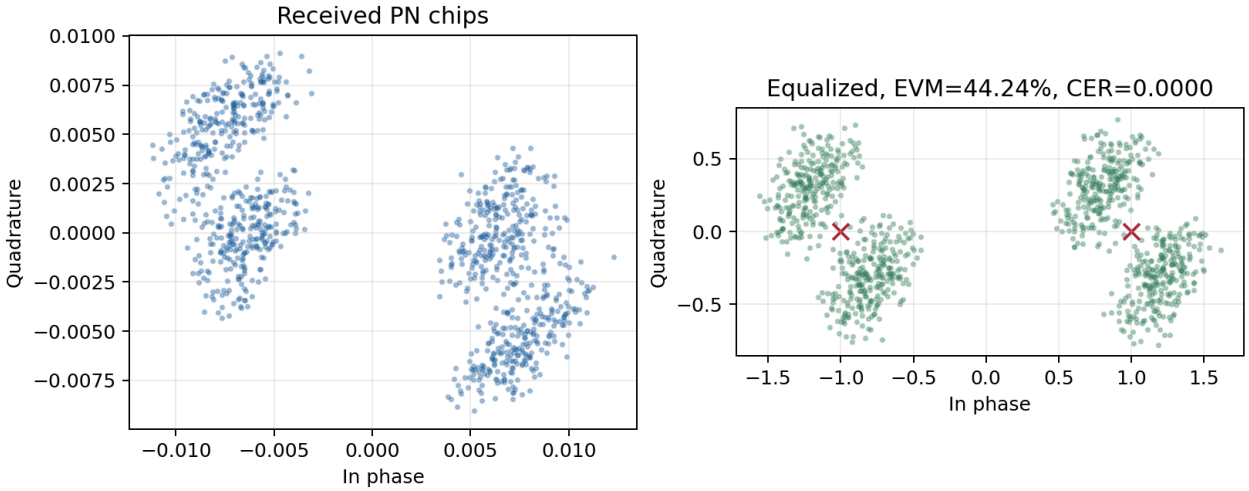
assert np.allclose(channel_estimate, record["channel"])
assert np.allclose(equalized, record["equalized"])
print("Noise power (dB):", record["noise_power_db"])
print("Active power (dB):", record["active_power_db"])
print(f"Reproducible Squelch Threshold (dB): {np.round((record['noise_power_db'] + record['active_p
print()
print("Correlation Peak Sample (start sample):", record["start_sample"])
print("CFO estimate (Hz):", record["cfo_hz"])
print("Complex Channel estimate:", channel_estimate)
print("magnitude and phase (degrees):", abs(channel_estimate), np.rad2deg(np.angle(channel_estimate
print("SNR, EVM, chip error rate:", record["snr_db"], record["evm"], record["chip_error_rate"])

for path in plot_real_pn_measurement(record, FIGURE_DIR):
    display(Image(filename=path))
```

```
Noise power (dB): -72.7383837465147
Active power (dB): -41.57082571550792
Reproducible Squelch Threshold (dB): -57.0

Correlation Peak Sample (start sample): 11020797
CFO estimate (Hz): 134.8701097106028
Complex Channel estimate: (0.005812607707387879-0.004945816536811875j)
magnitude and phase (degrees): 0.007632005606378102 -40.3937090496357
SNR, EVM, chip error rate: 31.16423759955298 0.4424354325100743 0.0
```





Report. Explain why the raw capture is retained even when a squelched file is available.

The raw capture preserves the complete received signal without applying squelch. Squelching can remove samples that fall below the selected threshold. Keeping the raw capture ensures that the original received data is still available.

Report. Explain why the raw branch is used for quantitative estimation.

The raw branch is used because squelching modifies the received signal based on its power. This modification could affect measurements such as correlation, CFO, channel estimation, and EVM. Using the raw samples ensures these estimates are calculated from the unmodified received signal.

Report. Explain what physical and radio effects are included in the end-to-end scalar coefficient.

The end-to-end scalar coefficient $\hat{h}$ represents the combined effect of the entire radio link on the transmitted signal. It includes the 0.08 transmit amplitude, transmitter and receiver gains, antenna effects, propagation through the channel, receiver scaling, and common phase shift. Therefore, $|\hat{h}|$ describes the overall amplitude scaling, while its phase describes the overall phase shift. It is not a calibrated measurement of path loss or voltage.

Lab 4 Task 4: Controlled Channel Experiments

Analyze four independent PN captures at each of three successfully acquired TX gains. Calculate coherent and magnitude averages, strength factors, and a gain-response regression.

In [5]:
```
from pathlib import Path
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import Image, display

def locate_lab_root():
    for base in (Path.cwd(), *Path.cwd().parents):
        if (base / "support" / "lab4_tools.py").is_file():
            return base
    raise FileNotFoundError("Open this notebook from its Lab 4 folder.")

LAB_ROOT = locate_lab_root()
sys.path.insert(0, str(LAB_ROOT / "support"))
WORK_DIR = LAB_ROOT / "work"
FIGURE_DIR = WORK_DIR / "figures"
WAVEFORM_DIR = WORK_DIR / "waveforms"
```

```
CAPTURE_DIR = Path(os.environ.get("ECE7377_LAB4_CAPTURE_DIR", WORK_DIR / "captures"))
for path in (FIGURE_DIR, WAVEFORM_DIR, CAPTURE_DIR):
    path.mkdir(parents=True, exist_ok=True)
print("Lab root:", LAB_ROOT)
print("Capture directory:", CAPTURE_DIR)
```

```
Lab root: /Users/marcpham/jupyter_notebook_files/Embedded-Wireless-Design-Labs/Lab4
Capture directory: /Users/marcpham/jupyter_notebook_files/Embedded-Wireless-Design-Labs/Lab4/work/c
aptures
```

Load and validate all captures

In [2]:
```
from lab4_tools import analyze_gain_sweep, plot_gain_sweep

gains = np.array([25, 32, 40])
runs = (1, 2, 3, 4)
result = analyze_gain_sweep(CAPTURE_DIR, gains=gains, runs=runs)
print("Loaded", len(result["records"]), "real captures")
for record in result["records"]:
    print(
        f"gain {record['tx_gain_db']:2d}, run {record['run']}: "
        f"|h|={record['channel_magnitude']:.8g}, "
        f"CFO={record['cfo_hz']:.2f} Hz, "
        f"EVM={100*record['evm']:.2f}%, CER={record['chip_error_rate']:.6f}"
    )
```

```
Loaded 12 real captures
gain 25, run 1: |h|=0.0076320056, CFO=134.87 Hz, EVM=44.24%, CER=0.000000
gain 25, run 2: |h|=0.0082823194, CFO=98.95 Hz, EVM=16.98%, CER=0.000000
gain 25, run 3: |h|=0.0082236939, CFO=118.20 Hz, EVM=20.28%, CER=0.000000
gain 25, run 4: |h|=0.0081891446, CFO=94.29 Hz, EVM=21.50%, CER=0.000000
gain 32, run 1: |h|=0.017117579, CFO=130.94 Hz, EVM=44.25%, CER=0.000000
gain 32, run 2: |h|=0.017728959, CFO=121.97 Hz, EVM=33.26%, CER=0.000000
gain 32, run 3: |h|=0.015542812, CFO=69.62 Hz, EVM=66.43%, CER=0.000000
gain 32, run 4: |h|=0.01695216, CFO=122.61 Hz, EVM=46.50%, CER=0.000000
gain 40, run 1: |h|=0.047935229, CFO=144.41 Hz, EVM=78.51%, CER=0.000000
gain 40, run 2: |h|=0.050741665, CFO=141.52 Hz, EVM=67.37%, CER=0.000000
gain 40, run 3: |h|=0.043021428, CFO=146.92 Hz, EVM=100.93%, CER=0.014663
gain 40, run 4: |h|=0.041325667, CFO=58.33 Hz, EVM=108.95%, CER=0.026393
```

Report. For every condition, report all four complex estimates h, |h|, CFO, EVM, chip error rate, and correlation metric.

In [7]:
```
records_df = pd.DataFrame([
    {
        "TX Gain (dB)": record["tx_gain_db"],
        "Run": record["run"],
        "Complex Channel Estimate h": record["channel"],
        "|h|": record["channel_magnitude"],
        "CFO (Hz)": record["cfo_hz"],
        "EVM (%)": 100 * record["evm"],
        "Chip Error Rate": record["chip_error_rate"],
        "Correlation (dB)": record["peak_to_median_db"],
    }
    for record in result["records"]
])

display(records_df.round({"|h|": 6, "CFO (Hz)": 2, "EVM (%)": 2, "Chip Error Rate": 6, "Correlatior
```

	TX Gain (dB)	Run	Complex Channel Estimate h	|h|	CFO (Hz)	EVM (%)	Chip Error Rate	Correlation (dB)
0	25	1	0.005813-0.004946j	0.007632	134.87	44.24	0.000000	57.65
1	25	2	-0.003761-0.007379j	0.008282	98.95	16.98	0.000000	59.03
2	25	3	0.008013-0.001849j	0.008224	118.20	20.28	0.000000	58.51
3	25	4	0.007458+0.003383j	0.008189	94.29	21.50	0.000000	59.76
4	32	1	0.003498+0.016756j	0.017118	130.94	44.25	0.000000	61.48
5	32	2	0.004917+0.017034j	0.017729	121.97	33.26	0.000000	61.40
6	32	3	-0.012056-0.009810j	0.015543	69.62	66.43	0.000000	59.85
7	32	4	0.005288+0.016106j	0.016952	122.61	46.50	0.000000	60.26
8	40	1	-0.037739-0.029556j	0.047935	144.41	78.51	0.000000	60.90
9	40	2	0.008649+0.049999j	0.050742	141.52	67.37	0.000000	59.86
10	40	3	-0.027374-0.033189j	0.043021	146.92	100.93	0.014663	69.38
11	40	4	-0.000386+0.041324j	0.041326	58.33	108.95	0.026393	69.06

In [10]:
```python
# Strength relative to 25 dB condition
reference_magnitude = result["summaries"][0]["mean_magnitude"]

summary_df = pd.DataFrame([
    {
        "TX Gain (dB)": summary["tx_gain_db"],
        "Complex Mean h": summary["complex_mean"],
        "|Complex Mean h|": summary["magnitude_of_mean"],
        "Mean |h|": summary["mean_magnitude"],
        "Strength Factor vs 25 dB": summary["mean_magnitude"] / reference_magnitude,
        "Strength Factor (dB)": 20 * np.log10(
            summary["mean_magnitude"] / reference_magnitude
        ),
    }
    for summary in result["summaries"]
])

display(summary_df)
```

	TX Gain (dB)	Complex Mean h	|Complex Mean h|	Mean |h|	Strength Factor vs 25 dB	Strength Factor (dB)
0	25	0.004380-0.002698j	0.005145	0.008082	1.000000	0.000000
1	32	0.000412+0.010021j	0.010030	0.016835	2.083125	6.374305
2	40	-0.014213+0.007145j	0.015907	0.045756	5.661616	15.058808

Coherent and magnitude averaging

In [3]:
```python
gain25_records = [record for record in result["records"] if record["tx_gain_db"] == 25]
channels = np.asarray([record["channel"] for record in gain25_records])

# Calculate the three different averages for the first condition.
complex_mean = np.mean(channels)
magnitude_of_mean = np.abs(complex_mean)
mean_magnitude = np.mean(np.abs(channels))

reference = result["summaries"][0]
assert np.allclose(complex_mean, reference["complex_mean"])
assert np.isclose(magnitude_of_mean, reference["magnitude_of_mean"])
assert np.isclose(mean_magnitude, reference["mean_magnitude"])

print("complex mean:", complex_mean)
```

```
print("magnitude of complex mean:", magnitude_of_mean)
print("mean magnitude:", mean_magnitude)
```

```
complex mean: (0.004380481236504305-0.0026977660242918002j)
magnitude of complex mean: 0.005144565811143782
mean magnitude: 0.008081790846431451
```

Report. For each gain, report the complex mean, magnitude of the complex mean, and mean magnitude.

In [8]:
```
summary_df = pd.DataFrame([
    {
        "TX Gain (dB)": summary["tx_gain_db"],
        "Complex Mean h": summary["complex_mean"],
        "|Complex Mean h|": summary["magnitude_of_mean"],
        "Mean |h|": summary["mean_magnitude"],
    }
    for summary in result["summaries"]
])

display(summary_df.round({"|Complex Mean h|": 6, "Mean |h|": 6}))
```

	TX Gain (dB)	Complex Mean h	|Complex Mean h|	Mean |h|
0	25	0.004380-0.002698j	0.005145	0.008082
1	32	0.000412+0.010021j	0.010030	0.016835
2	40	-0.014213+0.007145j	0.015907	0.045756

Report. Explain any difference between coherent and magnitude averaging. Explain why coherent averaging may be smaller when phase changes between independent captures.

Coherent averaging averages the complex channel estimates before taking the magnitude, so phase differences between captures can cause partial cancellation. This is visible in the results: at 25 dB, the magnitude of the complex mean was 0.00515, while the mean magnitude was 0.00808; at 40 dB, these values were 0.01591 and 0.04576, respectively. Each complex estimate has a magnitude and phase, so estimates with different phases point in different directions and partially cancel when averaged. Magnitude averaging avoids this cancellation by taking the magnitude of each estimate before averaging. Therefore, the larger difference at 40 dB indicates greater cancellation among the complex estimates due to their phase differences.

Strength factor and regression

Report. For each gain, report the strength factor. Report the 40-to-25 dB strength factor in linear and dB units. Report the regression slope, intercept, residuals, and R^2.

In [15]:
```
regression_df = pd.DataFrame({
    "TX Gain (dB)": x,
    "Strength Factor vs 25 dB (linear)": mean_magnitudes / mean_magnitudes[0],
    "Strength Factor vs 25 dB (dB)": 20 * np.log10(mean_magnitudes / mean_magnitudes[0]),
    "Residual (dB)": regression_residuals_db,
})

display(regression_df.round(4))

print(f"40-to-25 dB strength factor (linear): {strength_factor:.4f}")
print(f"40-to-25 dB strength factor (dB): {strength_factor_db:.2f} dB")
print(f"Regression slope: {regression_coefficient[0]:.4f}")
print(f"Regression intercept: {regression_coefficient[1]:.4f}")
print(f"R squared: {result['regression_r_squared']:.4f}")
```

	TX Gain (dB)	Strength Factor vs 25 dB (linear)	Strength Factor vs 25 dB (dB)	Residual (dB)
0	25	1.0000	0.0000	0.2319
1	32	2.0831	6.3743	-0.4348
2	40	5.6616	15.0588	0.2029

```
40-to-25 dB strength factor (linear): 5.6616
40-to-25 dB strength factor (dB): 15.06 dB
Regression slope: 1.0059
Regression intercept: -67.2281
R squared: 0.9975
```

In [16]:
```
mean_magnitudes = np.asarray([summary["mean_magnitude"] for summary in result["summaries"]])
mean_magnitudes_db = 20*np.log10(mean_magnitudes)

# Compare the high and low conditions and fit all three points.
strength_factor = mean_magnitudes[-1] / mean_magnitudes[0]
strength_factor_db = 20 * np.log10(strength_factor)

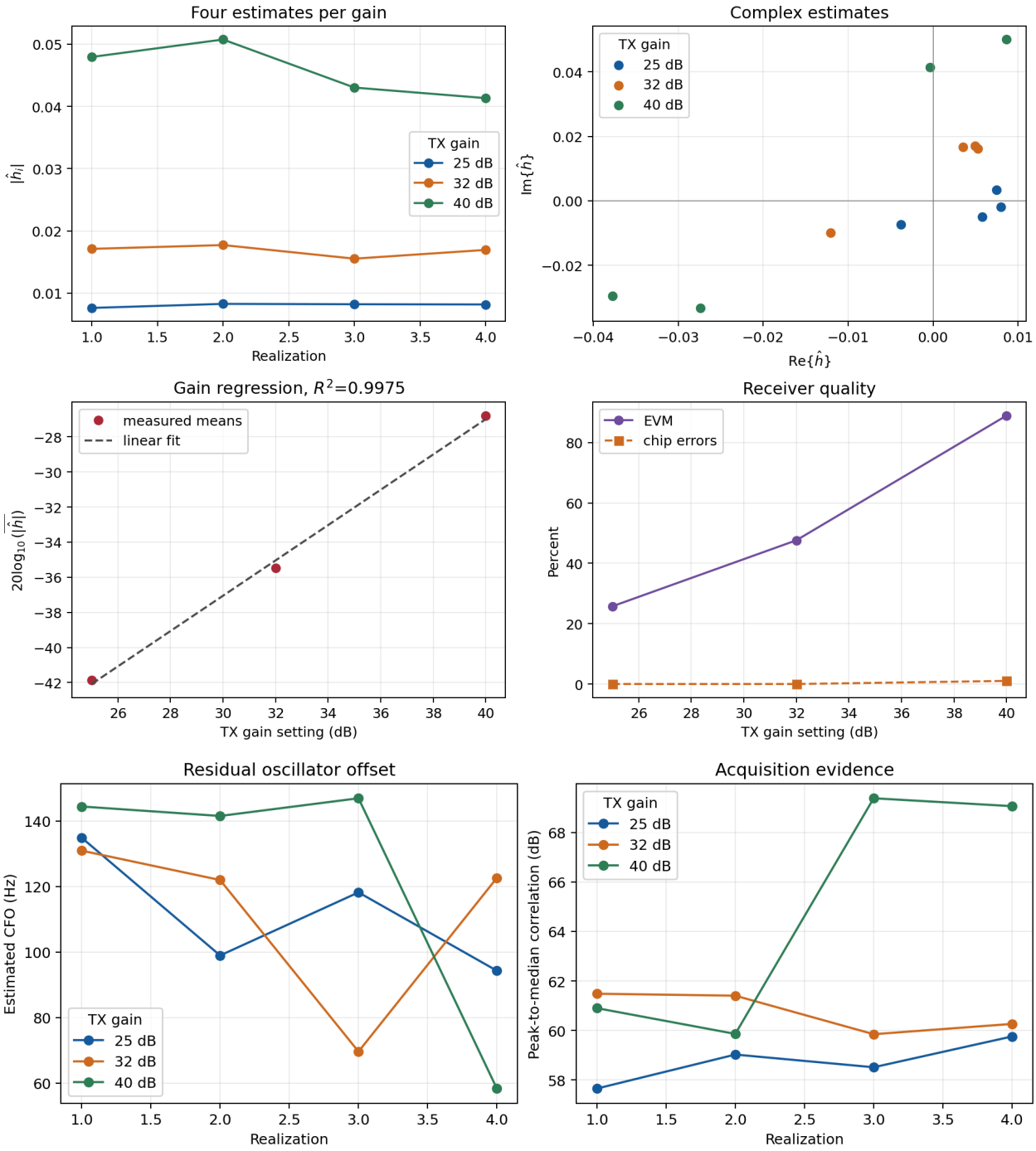
x = np.asarray([summary["tx_gain_db"] for summary in result["summaries"]])
regression_coefficient = np.polyfit(x, mean_magnitudes_db, 1)
regression_prediction_db = np.polyval(regression_coefficient, x)
regression_residuals_db = mean_magnitudes_db - regression_prediction_db

assert np.allclose(regression_coefficient, result["regression_coefficient"])

print("strength factor, linear and dB:", strength_factor, strength_factor_db)
print("slope and intercept:", regression_coefficient)
print("residuals (dB):", regression_residuals_db)
print("R squared:", result["regression_r_squared"])

for path in plot_gain_sweep(result, FIGURE_DIR):
    display(Image(filename=path))
```

```
strength factor, linear and dB: 5.6616161017495825 15.058808354823245
slope and intercept: [  1.00585292 -67.22805447]
residuals (dB): [ 0.2318836  -0.43478176  0.20289815]
R squared: 0.9975149689559849
```



Report. State every parameter held fixed and label TX gain as a controlled link-strength proxy.

The RX gain, center frequency, sample rate, radio pair, antenna arrangement, PN waveform, and receiver processing were held fixed across all measurements. Only the TX gain was varied between 25, 32, and 40 dB, providing a controlled and repeatable way to change the strength of the received link. Therefore, TX gain was used as a controlled link-strength proxy, meaning it represents intentional changes in link strength while the rest of the setup remains unchanged. It is only a proxy for link strength and does not represent a change in physical distance.

Report. Interpret the gain, frequency, and distance or proxy results without claiming that normalized values are calibrated power.

A fixed-rack setup was used, so the TX and RX positions remained unchanged and TX gain served as the controlled link-strength proxy. As TX gain increased from 25 to 32 to 40 dB, the mean channel magnitude increased from 0.00808 to 0.01684 to 0.04576, showing a clear increase in the measured end-to-end link strength. Because the

radio positions were fixed, these results represent the effect of changing TX gain rather than changing physical distance. The measured channel magnitudes are normalized end-to-end coefficients that include the radio gains, antennas, propagation, and receiver scaling, so they should not be interpreted as calibrated received power or path loss. A frequency comparison was not performed because measurements were only conducted at 2.45 GHz with the assigned configuration because the second band 5.8 GHz was not approved.

Conclusion

Report. The report must include a statement connecting timing recovery, phase correction, known training, least squares, and equalization.

Overall, this lab demonstrated how several receiver operations work together to reliably recover transmitted data. Timing recovery first aligns the receiver with the correct sampling instant so that symbols can be observed with minimal interference from neighboring symbols. Once timing is established, known training symbols and pilots provide references for identifying and correcting phase errors and other channel effects. These known symbols allow the receiver to use least-squares estimation to determine the complex channel coefficient h^\hat{h}h^, which captures the link's amplitude scaling and phase shift. The resulting estimate is then used for equalization, where the received symbols are compensated for the estimated channel distortion. After equalization, the symbols move closer to their expected constellation locations, making correct detection more reliable. Together, timing recovery, phase correction, known training, least-squares estimation, and equalization form a connected process that progressively removes impairments from the received signal.